

Listing of the source code (written in Pascal)

```

program cones(input,output);

const
  maxN = 20; maxdim = 10;
type
  pmatrix = array[1..2,1..100] of real;
var
  name: packed array[1..12] of char;
  infile, outfile: text;
  i, j, zij, dim, N, count, Ndata, gliy: integer;
  seed: integer:= 1959; gliset, totalcount: integer:= 0;
  glir: array[1..97] of integer;
  change: array[1..maxN] of boolean;
  z: array[1..maxN,1..maxN] of boolean;
  Xnew, Xold: array[1..maxN,1..maxdim] of real:= zero;
  Rnew, Rold: array[1..maxN] of real:= zero;
  E: array[1..100] of real:= zero;
  Enew, Eold: array[1..maxN,1..maxN] of real:= zero;
  deltaE, Rave, R, Rmin, T, Eave, Evar, spheat, glgset: real;
  data: pmatrix;

#include 'rand.pas'

procedure energy;
const
  roottwo = sqrt(2);
var
  Rij, Xij, s2: real;
  i, j, k: integer;
begin
  deltaE:= 0;
  for i:=1 to N-1 do
    for j:=i+1 to N do
      if change[i] or change[j] then begin
        Rij:= Rnew[i]-Rnew[j];
        Xij:= 0;
        for k:=1 to dim do Xij:=Xij+(Xnew[i,k]-Xnew[j,k])**2;
        s2:= -(Rij**2)+Xij;
        Xij:= sqrt(Xij);
        if z[i,j] then
          if s2>0 then Enew[i,j]:=(Xij+Rij)/(roottwo*Rave)
          else
            if Rij>0 then Enew[i,j]:=sqrt(s2+2*(Rij**2))/Rave
            else Enew[i,j]:= 0
        else
          if s2>0 then Enew[i,j]:= 0
          else Enew[i,j]:= (abs(Rij)-Xij)/(roottwo*Rave);
        deltaE:= deltaE+Enew[i,j]-Eold[i,j]
      end
  end; { energy }

```

```

procedure reconfigure;
var
  i, k: integer;
  Efraction, norm: real;
  displacement: array[1..maxdim+1] of real;
begin
  Rave:= R;
  for i:=1 to N do begin
    Efraction:= 2*Eold[i,i]/E[1];
    if ran2(seed)<Efraction then change[i]:= true;
    repeat
      repeat
        norm:= 0;
        for k:=1 to dim+1 do begin
          displacement[k]:= 2*ran2(seed)-1;
          norm:= norm+displacement[k]**2
        end
      until norm<1;
      norm:= Eold[i,i]*R*gasdev(seed)/sqrt(norm);
      for k:=1 to dim do
        Xnew[i,k]:= Xold[i,k]+displacement[k]*norm;
      Rnew[i]:= Rold[i]+displacement[dim+1]*norm
    until Rnew[i]>0;
    Rave:= Rave+(Rnew[i]-Rold[i])/N
  end
end; { reconfigure }

procedure update;
var
  i, j, k: integer;
begin
  for i:=100 downto 2 do E[i]:= E[i-1];
  E[1]:= 0;
  Rmin:= Rnew[1];
  for i:=1 to N do begin
    if Rnew[i]<Rmin then Rmin:= Rnew[i];
    if change[i] then begin
      for k:=1 to dim do Xold[i,k]:= Xnew[i,k];
      Rold[i]:= Rnew[i]
    end;
    for j:=i+1 to N do
      if change[i] or change[j] then Eold[i,j]:= Enew[i,j];
    Eold[i,i]:= 0;
    for j:=1 to i-1 do Eold[i,i]:= Eold[i,i]+Eold[j,i]/2;
    for j:=i+1 to N do Eold[i,i]:= Eold[i,i]+Eold[i,j]/2;
    E[1]:= E[1]+Eold[i,i]
  end;
  for i:=1 to N do begin
    for k:=1 to dim do Xold[i,k]:= Xold[i,k]/Rmin;
    Rold[i]:= Rold[i]/Rmin
  end;
  Rave:= Rave/Rmin;
  R:= Rave
end; { update }

```

```

procedure statistics;
var
  i: integer;
  E2: real;
begin
  Eave:= 0;
  E2:= 0;
  for i:=1 to 100 do begin
    Eave:= Eave+E[i];
    E2:= E2+E[i]**2
  end;
  Eave:= Eave/100;
  E2:= E2/100;
  Evar:= sqrt(E2-(Eave**2));
  if totalcount=0 then T:= 100;
  if T>0 then spheat:= (Evar/T)**2;
  totalcount:= totalcount+100
end; { statistics }

procedure startup;
begin
  write('Input data file: ');
  readln(name);
  open(infile,name,old);
  reset(infile);
  open(outfile,'cone.out');
  rewrite(outfile);
  readln(infile,N);
  for i:=1 to N-1 do
    for j:=i+1 to N do begin
      read(infile,zij);
      if zij=0 then z[i,j]:= false
      else
        if zij=1 then z[i,j]:= true
        else writeln('wrong data in input file')
    end;
  write('how many space dimensions? ');
  readln(dim);
  for i:=1 to N do begin
    change[i]:= true;
    for j:=1 to dim do Xnew[i,j]:= 0;
    Rnew[i]:= i+1
  end;
  Rave:= (N+4)/2;
  energy;
  update;
  writeln('Energy of initial configuration: ',E[1]:9);
  writeln(outfile,
    'Energy of initial configuration: ',E[1]:9)
end; { startup }

```

```

procedure warmup;
begin
  count:= 0;
  while (count<100) and (E[1]>0) do begin
    reconfigure;
    energy;
    update;
    count:= count+1;
    writeln(count)
  end;
  statistics;
  writeln('
    Count    Temp.    E[1].    Eave.    Evar.    spheat');
  writeln('
    warmup -----',E[1]:8,Eave:8,Evar:8,spheat:8);
  writeln(outfile,'
    Count    Temp.    E[1].    Eave.    Evar.    spheat');
  writeln(outfile,'
    warmup -----',E[1]:8,Eave:8,Evar:8,spheat:8);
  write('waiting for <cr> ... ');
  readln
end; { warmup }

procedure decide;
begin
  if deltaE<0 then begin
    update;
    count:= count+1
  end
  else
    if T>0 then
      if ran2(seed)<4*exp(-deltaE/T) then begin
        update;
        count:= count+1
      end
end; { decide }

procedure anneal;
var
  continue: char;
begin
  Ndata:= 0;
  continue:= 'y';
  while continue='y' do begin
    count:= 0;
    while (count<100) and (E[1]>0) do begin
      reconfigure;
      energy;
      decide
    end;
    statistics;
    writeln('
      totalcount:5,T:8,E[1]:8,Eave:8,Evar:8,spheat:8);
    writeln(outfile,'
      totalcount:5,T:8,E[1]:8,Eave:8,Evar:8,spheat:8);
  end;
end;

```

```

Ndata:= Ndata+1;
data[1,Ndata]:= T;
data[2,Ndata]:= Eave;
write('y/t/n: ');
readln(continue);
if continue='t' then begin
  write('new T: ');
  readln(T);
  continue:= 'y'
end;
if continue='y' then T:= T*0.9;
if Ndata=35 then continue:= 'n'
end
end; { anneal }

{ procedure prepareplot(Ndata: integer; data: pmatrix);
fortran; }

procedure writeout(var where: text);
var
  i, j, k: integer;
begin
  writeln(where);
  writeln(where,'Incidence matrix:');
  writeln(where);
  for i:=1 to N do begin
    for j:=1 to N do write(where,' ',z[i,j]:1);
    writeln(where)
  end;
  writeln(where);
  writeln(where,'dimension: ',dim:2);
  writeln(where);
  for i:=1 to N do begin
    write(where,'i=',i:2,', X=');
    for k:=1 to dim do write(where,' ',Xold[i,k]:8);
    writeln(where,' R=',Rold[i]:8)
  end;
  writeln(where);
  writeln(where,'Energy matrix:');
  writeln(where);
  for i:=1 to N do begin
    for j:=1 to N do write(where,' ',Eold[i,j]:8);
    writeln(where)
  end;
  writeln(where);
  writeln(where,'Total energy: ',E[1])
end; { writeout }

begin { main program }
  startup;
  warmup;
  anneal;
  { prepareplot(Ndata,data); }
  writeout(output);
  writeout(outfile)
end.

```